

Maximal Number of Equilibria in Product Two-Action Games

Candidate Number: 69729

**A Dissertation submitted to the Department of Mathematics
of the London School of Economics and Political Science**

March 30, 2026

Summary

Finding Nash equilibria is a central problem in the study of strategic interaction. A natural question is how many equilibria a game can have. Hertling and Vujic (2024) tackle this question by establishing a bound on the maximal number of equilibria for an n -player game where each player has two pure strategies and a restricted utility function. We study this bound and provide a more accessible proof in this dissertation. We argue directly via the best-response condition and provide greater clarity on the main ideas behind the proof. A computational implementation that verifies these equilibria for arbitrary n is also provided.

Contents

Summary	ii
1 Introduction	1
2 Strategic Games	3
2.1 Finite Game	3
2.1.1 Mixed Extension	3
2.1.2 Best Response Condition	4
2.2 Derangements	4
3 Product-Action Games	6
3.1 Equilibrium Properties	7
3.1.1 Mixing Players	7
3.1.2 Pure Players	8
3.1.3 Maximal Bound	10
4 Conditions for Maximality	11
4.1 Illustrative Example	11
4.1.1 Case 1: Bound Is Achieved	11
4.1.2 Case 2: Bound Is Not Achieved	12
4.2 Parameter Choice	13
4.2.1 Delta Sort Order	13
4.3 Sort-Order Proof	14
4.3.1 π Notation	14
4.3.2 Parameter Specification	15
4.3.3 Case Analysis and Main Result	15
5 Conclusion	20
Bibliography	21
A	23
A.1 Code	23
A.2 Proofs	25
A.2.1 Case 2 Cardinality	25

Chapter 1

Introduction

The study of equilibria in strategic-form games is a central topic in game theory. The existence of at least one equilibrium in any finite strategic game was proved by Nash (1951). A natural follow-up question is what the structure of equilibria looks like. One answer is the Lemke–Howson algorithm. Lemke and Howson (1964) provide a method for finding an equilibrium, but it does not guarantee finding all equilibria, which leads to a more fundamental question: how many equilibria can a game have?

For $n \times n$ non-degenerate bimatrix games, Shubik and Quint (1998) conjectured that the maximum number of Nash equilibria is $2^n - 1$. This was confirmed for $n \leq 4$ by Keiding (1997) and McLennan and Park (1999), and recently for $n = 5$ by Ickstadt, Theobald, and von Stengel (2025), but it was shown to be false for $n \geq 6$ by von Stengel (1999). Beyond the two-player setting, for the special case where every player assigns positive probability to each of their strategies (totally mixed Nash equilibria), McKelvey and McLennan (1997) established an upper bound on the number of such equilibria.

However, the question of the maximal number of equilibria in an n -player game remains largely open. To help remedy this gap, Hertling and Vujic (2024) analyse the equilibrium structure of n -player games in which each player has exactly two strategies. The authors restrict attention to product two-action games, a class of games characterised by a special multiplicative structure on the utility functions, and establish an upper bound on the maximal number of equilibria. This bound is

$$\sum_{l=1}^n \binom{n}{l} 2^{l-1} !(n-l) + !n, \quad (1.1)$$

where n is the number of players and l is the number of players playing pure strategies, and $!n$ is the number of *derangements* (permutations without fixed points) of n elements. Hertling and Vujic’s paper represents a first step towards finding an upper bound in the general n -player case. While the result is compelling, we believe the paper in its current form is difficult to follow due to heavy notation and insufficient exposition of the main ideas behind the proof. Hertling and Vujic begin

by establishing the framework of a finite game and its mixed extension. We believe this generality is not needed for the primary results; instead, our dissertation uses a simplified construction that avoids the use of vector spaces. To increase clarity on the class of games under discussion, our dissertation introduces product two-action games directly through a definition of the utility function, rather than the payoff difference function used by Hertling and Vujic. The utility function for player i in $[n] = \{1, 2, \dots, n\}$ is

$$u^i(x) = u^i(x_1, \dots, x_n) = x_i \prod_{j=1, j \neq i}^n (x_j - a_j^i), \quad (1.2)$$

where $x = (x_1, \dots, x_n)$ is a strategy profile and $a_j^i \in (0, 1)$ is a parameter. Their paper also includes a (-1) term in the utility function; we show that the results hold without it.

A recurring technique in the proofs of Hertling and Vujic is to determine the sign of the utility function by counting the number of positive and negative terms. To do this systematically, the authors introduce what they call an *increment* function, which can be thought of as a sign-counting device. While the increment function provides a systematic procedure, what it gains in formalism it loses in clarity. In our dissertation, we circumvent this increment function and instead argue directly through the best-response condition of Nash equilibria. Hertling and Vujic show that a specific ordering of the parameters a_j^i achieves the bound. To simplify the proof, we specify the parameters explicitly as $a_j^i = [j - i]_n / n$ (where $[k]_n = k \bmod n$) and show that this achieves the upper bound. Our proof (Theorem 4.4) is more accessible due to a finer breakdown of the case structure, which provides greater intuition behind the key ideas. We also directly construct the Nash equilibria rather than operating through an increment function, and provide a computational implementation (Appendix A.1) that generates and verifies these equilibria, for any n .

Throughout this dissertation, each chapter opens with a brief discussion of how our approach departs from that of Hertling and Vujic (2024), and the motivation behind each choice. The remainder of this dissertation is organised as follows. Chapter 2 introduces strategic games, their mixed extension, and the combinatorial notion of derangements. Chapter 3 defines product two-action games and establishes structural properties of their equilibria, including the connection between derangements and mixing players (the players that mix both strategies with positive probabilities and therefore have to be indifferent between them), and a parity condition governing pure players. Chapter 4 contains the main results: we motivate the need for a careful choice of parameters through illustrative examples, introduce (following Hertling and Vujic) the delta sort order, specify the parameters explicitly (which seems new), and prove that every permutation yields a Nash equilibrium. Chapter 5 concludes with a discussion of the results and possible directions for future work.

Chapter 2

Strategic Games

In this chapter, the standard concepts of finite games and their mixed extension are introduced alongside the combinatorial notation used throughout. Our treatment is tailored to two-action games from the outset. Rather than setting up the general framework of vector spaces and affine subspaces used by Hertling and Vujic, we work directly with strategy variables $x_i \in [0, 1]$, which simplifies the notation considerably.

2.1 Finite Game

A strategic game is defined by a finite set of players and, for each player, a finite set of available strategies. We let the set of players be $[n] = \{1, \dots, n\}$. For each player $i \in [n]$, let $S^i = \{s_0^i, \dots, s_{n_i}^i\}$ denote the set of pure strategies of player i .

The set of pure strategy profiles is $S = S^1 \times \dots \times S^n$. For each $i \in [n]$, the utility function of player i is $u^i : S \rightarrow \mathbb{R}$, and the pure strategy profiles are given by tuples $(s_{j_1}^1, \dots, s_{j_n}^n) \in S$, with $(j_1, \dots, j_n) \in J := \prod_{i \in [n]} \{0, 1, \dots, n_i\}$. Since we restrict attention to games in which each player has exactly two strategies, the pure strategy of player $i \in [n]$ can be represented by a single variable $s_i \in \{0, 1\}$, where $s_i = 0$ means the player plays their first strategy and $s_i = 1$ means they play their second strategy. Then the set of all pure-strategy profiles $s = (s_1, \dots, s_n)$ becomes the set of corners $\{0, 1\}^n$ of the n -dimensional cube $[0, 1]^n$; the cube itself represents the set of mixed strategies of the players, considered next.

2.1.1 Mixed Extension

A mixed strategy for player i is a probability distribution over their two pure strategies. Since there are only two pure strategies, a mixed strategy is determined by a single number $x_i \in [0, 1]$, the probability of playing strategy 1. The strategy set for player i is thus extended from $\{0, 1\}$ to $[0, 1]$. A mixed strategy profile is a vector $x = (x_1, \dots, x_n) \in [0, 1]^n$. When players randomise independently, the utility for

player i is given by the expected payoff over the pure strategy profiles. We define L as the set of players who play pure strategies, so that $||[n] \setminus L||$ players mix.

2.1.2 Best Response Condition

We follow the same construction as seen in (von Stengel, 2022, Definition 3.1), and start by considering the partial profile

$$s_{-i} = (s_1, \dots, s_{i-1}, s_{i+1}, \dots, s_n), \quad (2.1)$$

consisting of the strategies of all players $j \neq i$. For any strategy $s_i \in S_i$, this partial profile is extended to a full strategy profile by $s = (s_i, s_{-i})$. Then s_i is a best response to s_{-i} if

$$u_i(s_i, s_{-i}) \geq u_i(\bar{s}_i, s_{-i}) \quad \text{for all } \bar{s}_i \in \{0, 1\}. \quad (2.2)$$

An equilibrium of the game is a strategy profile s in which each strategy is a best response to the others. That is, (2.2) holds for all players $i = 1, \dots, n$. We now state the best-response condition for the mixed extension, taken from (von Stengel, 2022, Proposition 6.3, p. 148).

Lemma 2.1 (Best-response condition). *Let $x_i \in [0, 1]$ be a mixed strategy of player i and x_{-i} a partial profile of the other players. Let*

$$v_i = \max\{u_i(0, x_{-i}), u_i(1, x_{-i})\}$$

be the best possible expected payoff against x_{-i} for a pure strategy of player i . Then x_i is a best response to x_{-i} if and only if every pure strategy played with positive probability achieves payoff v_i . That is,

$$x_i > 0 \implies u_i(1, x_{-i}) = v_i, \quad 1 - x_i > 0 \implies u_i(0, x_{-i}) = v_i.$$

In particular, when player i mixes, i.e. $x_i \in (0, 1)$, both pure strategies have positive probability, so $u_i(0, x_{-i}) = u_i(1, x_{-i})$.

In other words, to check whether a mixed strategy profile is a Nash equilibrium, we do not need to consider all possible mixed strategies. Instead, it suffices to check that each player assigns positive probability only to pure strategies that are best responses.

2.2 Derangements

We take a brief detour from finite games to introduce combinatorial concepts that will appear in the subsequent analysis. The collection of all permutations of a set forms a group called the symmetric group of that set. So, a permutation can be described by $\pi \in S_n$ where S_n denotes the symmetric group on $\{1, \dots, n\}$.

Definition 2.2. A permutation $\pi \in S_n$ is called a *derangement* if $\pi(i) \neq i$ for all $i \in \{1, \dots, n\}$.

Definition 2.3. An index $i \in [n]$ is called a *fixed point* of π if $\pi(i) = i$, and we define the set of fixed points by $F(\pi) := \{i \in [n] : \pi(i) = i\}$.

Lemma 2.4. (Stanley, 2011, p. 227) The number $!n$ of derangements is given by

$$!n = \left\lfloor \frac{n!}{e} \right\rfloor$$

where $\lfloor x \rfloor$ is the nearest integer function (rounding up or down).

Proof. This is a standard proof by inclusion-exclusion. The number of all possible arrangements is $n!$. We then remove all cases in which some element in $[n]$ maps to itself, giving us $n! - \binom{n}{1}(n-1)!$, but when subtracting $\binom{n}{1}(n-1)!$, we do not distinguish whether elements other than the chosen fixed point i are also fixed. A permutation with fixed points i and j is therefore subtracted twice. So we have that $\binom{n}{2}(n-2)!$ permutations have been over-subtracted and we must therefore add $\binom{n}{2}(n-2)!$. We continue this inclusion-exclusion argument to arrive at

$$\begin{aligned} !n &= n! - \binom{n}{1}(n-1)! + \binom{n}{2}(n-2)! - \dots + (-1)^n \binom{n}{n}(n-n)! \\ &= n! \left(1 - \frac{1}{1!} + \frac{1}{2!} - \frac{1}{3!} + \dots + (-1)^n \frac{1}{n!} \right) \\ \Rightarrow \left| !n - \frac{n!}{e} \right| &= \frac{1}{n+1} - \frac{1}{(n+1)(n+2)} + \frac{1}{(n+1)(n+2)(n+3)} - \dots < \frac{1}{2}. \end{aligned}$$

Since $!n$ is an integer, it follows that $!n$ lies within a distance of $\frac{1}{2}$ of the real number $\frac{n!}{e}$.

However, there can be at most one integer within a distance of $\frac{1}{2}$ of any real number. Therefore, $!n$ is the unique nearest integer to $\frac{n!}{e}$. $\square$

Chapter 3

Product-Action Games

We now define product two-action games and establish structural properties of their equilibria. Hertling and Vujic define this class of games through the payoff difference function λ^i and include a sign vector $(-1)^{v_i}$ in the utility function. We instead define the game directly through the utility function u^i in (1.2) and show that the results hold without the sign term. We also replace the increment function with the best-response condition of Theorem 3.3, which we find more transparent.

Definition 3.1 (Product two-action game). A strategic game is a *product two-action game* if the utility function of each player i is written as

$$u^i(x) = u^i(x_1, \dots, x_n) = x_i \prod_{j=1, j \neq i}^n (x_j - a_j^i) . \quad (3.1)$$

where $a_j^i \in (0, 1)$ and $x = (x_1, \dots, x_n) \in [0, 1]^n$ is a mixed strategy profile.

To ensure our game is not degenerate, we add the additional constraint $a_j^{i_1} \neq a_j^{i_2}$ for all distinct $i_1, i_2 \in [n]$ and all $j \notin \{i_1, i_2\}$. This condition ensures non-degeneracy, as otherwise some player could make players i_1 and i_2 indifferent, allowing one of these players to play any strategy in $[0, 1]$.

Definition 3.2. We define the **partial utility function** as $Q_i : S^* \rightarrow \mathbb{R}$ where $S^* = S^1 \times \dots \times S^{i-1} \times S^{i+1} \times \dots \times S^n$ and

$$Q_i(x_{-i}) = \prod_{j=1, j \neq i}^n (x_j - a_j^i)$$

Remark 1. Player i 's payoff is zero when they play their first strategy ($x_i = 0$). Any game can be normalised to have this property without changing its equilibria. This is justified because in any game, only the payoff differences between the pure strategies matter for equilibria (von Stengel, 2022, Proposition 9.7, p. 240).

Remark 2. The product $Q_i(x_{-i}) = \prod_{j \neq i} (x_j - a_j^i)$ is the payoff to player i 's second strategy. If that product is positive, then player i prefers to play $x_i = 1$. If it is

negative, then player i prefers to play $x_i = 0$, and only if it is zero then player i can mix with $0 \leq x_i \leq 1$.

Remark 2 has an important consequence: it yields a necessary and sufficient condition that any equilibrium must satisfy. We now formalise this condition in the following theorem.

Theorem 3.3. *A strategy profile x is a Nash equilibrium if and only if, for every $i \in [n]$,*

$$x_i = \begin{cases} 1 & \text{if } Q_i(x_{-i}) > 0, \\ 0 & \text{if } Q_i(x_{-i}) < 0, \\ \text{any value in } [0, 1] & \text{if } Q_i(x_{-i}) = 0. \end{cases} \quad (3.2)$$

Proof. ($\Rightarrow$) Assume that x is a Nash equilibrium. Since $u_i(x) = x_i \cdot Q_i(x_{-i})$, we have $u_i(1, x_{-i}) = Q_i(x_{-i})$ and $u_i(0, x_{-i}) = 0$. By Lemma 2.1, x_i is a best response to x_{-i} if and only if every pure strategy played with positive probability achieves the maximum payoff $v_i = \max\{0, Q_i(x_{-i})\}$.

If $Q_i(x_{-i}) > 0$, then $v_i = Q_i(x_{-i}) > 0$, so only strategy 1 achieves v_i , and hence $x_i = 1$. If $Q_i(x_{-i}) < 0$, then $v_i = 0$, so only strategy 0 achieves v_i , and hence $x_i = 0$. If $Q_i(x_{-i}) = 0$, then both strategies achieve $v_i = 0$, so any $x_i \in [0, 1]$ is a best response.

($\Leftarrow$) We need to show that each player is best responding when playing according to (3.2), but by Remark 2 we know that the player's preference exactly matches up with the strategy given in (3.2). $\square$

3.1 Equilibrium Properties

To understand when the maximal bound (1.1) on possible equilibrium numbers is achieved, we first establish some structural properties of equilibria. We treat the cases of mixed and pure players separately for clarity.

3.1.1 Mixing Players

We first note that for an n player product two-action game with each player $i \in [n]$ playing a strategy $x_i \in (0, 1)$, i.e., a totally mixed Nash equilibrium (TMNE), we have that the maximum number of equilibria is given by the number of derangements $!n$. This follows from the special utility function, as for some player i the utility function (3.1) u^i cannot be made 0 with constraint $x_i \in (0, 1)$ unless some player $j \neq i$ plays a strategy $x_j = a_j^i$. Now, for some permutation π , this condition means we require player $\pi(i)$ in $[n]$ to play $x_{\pi(i)} = a_{\pi(i)}^i$ with condition $\pi(i) \neq i$; this is exactly the definition of a derangement. As any one derangement describes one of these equilibria, the set of all of them describes all possible equilibria.

Remark 3. Now consider an equilibrium x , where the subset L of $[n]$ is the set of players who play a pure strategy, and $[n] \setminus L$ is the set of other players who play a mixed strategy. We know that the payoff to the players in $[n] \setminus L$ must be zero, and that the payoff to these mixing players due to the special payoff function (3.1) is independent of the pure strategies of the players in L , as when $u^i(x) = 0$ it does not matter whether the pure players play 0 or 1. So for some specific pure strategy profile, the Nash equilibrium condition for the mixing players is achieved by any derangement π of $[n] \setminus L$ with player $\pi(i)$ in $[n] \setminus L$ playing $x_{\pi(i)} = a_{\pi(i)}^i$ as described so that $u^i(x) = 0$, for all i in $[n] \setminus L$.

3.1.2 Pure Players

Having established the connection between derangements and the number of players mixing, we move on to making some observations on the structure of pure players in an equilibrium.

Lemma 3.4. *Let $L \subseteq [n]$ and $x = (x_1, \dots, x_n) \in [0, 1]^n$ where $x_i \in \{0, 1\}$ for $i \in L$ and $0 < x_i < 1$ for $i \notin L$ (exactly the players in L play pure strategies).*

- (a) *If $L = [n]$, then x is a Nash equilibrium if and only if $|\{i \in [n] \mid x_i = 0\}|$ is even.*
- (b) *If $L = [n] \setminus \{i\}$, then x is not a Nash equilibrium.*
- (c) *If $L = \{i\}$, then x is a Nash equilibrium if and only if the players other than i play a completely mixed equilibrium via a derangement of $[n] \setminus \{i\}$, and $x_i = 0$ or $x_i = 1$ is chosen as the respective best response.*
- (d) *Suppose that x is a Nash equilibrium, and let $y = (y_1, \dots, y_n)$ such that $y_i = x_i$ for $i \in [n] \setminus L$ and $y_i \in \{0, 1\}$ for $i \in L$. Then y is a Nash equilibrium if and only if $|\{i \in L \mid y_i \neq x_i\}|$ is even.*

Proof. To prove (a) we first define $R = \{i \in [n] : x_i = 0\}$ and $S = \{i \in [n] : x_i = 1\}$, (note $R \cup S = [n]$), then our partial utility function Q_i is

$$Q_i(x_{-i}) = \prod_{j \in R \setminus \{i\}} (0 - a_j^i) \prod_{j \in S \setminus \{i\}} (1 - a_j^i) \Rightarrow \text{sign}(Q_i(x_{-i})) = -1^{|R \setminus \{i\}|}.$$

By Theorem 3.3, we know that the best response depends on the sign of Q_i . Given some equilibrium, we take a player $i \in R$, i.e., a player whose strategy is to play $x_i = 0$. This implies $Q_i(x_{-i}) < 0$, consequently, we require $|R| - 1$ to be odd, hence $|R|$ must be even. Now if we take player $i \in S$ then we require $Q_i(x_{-i}) > 0$ and so $|R|$ must be even.

To prove (b), notice this statement is equivalent to saying that all players play pure strategies except player i . However, as each player $j \neq i$ is playing either 0

or 1, there is no way to make player i 's utility function 0, so player i is not best responding.

(c) The players in $[n] \setminus \{i\}$ must mix within themselves, but this is exactly the case of Remark (3) with $n - 1$ players instead of n . So the equilibrium can be defined via a derangement of $[n] \setminus \{i\}$. Now the strategy of player i can be found by checking the sign of the partial utility function Q_i , which cannot be 0 due to our assumption, and so is uniquely defined as either 0 or 1.

For (d) We start with some Nash equilibrium x and we take some strategy profile y with property $y_i = x_i$ for $i \in [n] \setminus L$ and $y_i \in \{0, 1\}$ for $i \in L$. We define $R_x = \{i \in [n] : x_i = 0\}$ and $R_y = \{i \in [n] : y_i = 0\}$. Now the partial utility function Q_i for some player $i \in L$ is,

$$Q_i(x_{-i}) = \prod_{j \in L \setminus \{i\}} (x_j - a_j^i) \prod_{j \in [n] \setminus L} (x_j - a_j^i)$$

Similarly for y we have

$$Q_i(y_{-i}) = \prod_{j \in L \setminus \{i\}} (y_j - a_j^i) \prod_{j \in [n] \setminus L} (y_j - a_j^i).$$

$$\Rightarrow \text{sign} \left(\frac{Q_i(x_{-i})}{Q_i(y_{-i})} \right) = \text{sign} \left(\frac{\prod_{j \in L \setminus \{i\}} (x_j - a_j^i)}{\prod_{j \in L \setminus \{i\}} (y_j - a_j^i)} \right) = \frac{-1^{|R_x \setminus \{i\}|}}{-1^{|R_y \setminus \{i\}|}} = -1^{|R_x \setminus \{i\}| - |R_y \setminus \{i\}|}$$

Now, for each player i playing a pure strategy in y , we either have that $y_i = x_i$ or $y_i \neq x_i$. We assume w.l.o.g that $|R_x| \geq |R_y|$. Since x is an equilibrium, we know x_i is a best response to the sign($Q_i(x_{-i})$). If $y_i = x_i$ then all we need to check is that the sign of $Q_i(y_{-i})$ matches that of $Q_i(x_{-i})$. This is equivalent to the condition $|R_x \setminus \{i\}| - |R_y \setminus \{i\}| = |R_x| - |R_y| = 2k$, $k \in \mathbb{N}$. Now if $x_i \neq y_i$ then we require that y_i be the opposite sign of x_i so $|R_x \setminus \{i\}| - |R_y \setminus \{i\}| = 2k + 1$. But we also know that x and y differ at the i th position, and so our condition becomes equivalent to $|R_x| - |R_y| = 2k$. Finally, we note that asking how many more zeros are in x is the same as asking in how many terms do x and y differ in, as the only way x has more zeros than y is if y_i takes the value 1 for those i such that $x_i = 0$. Now we require the number of 0s that differ between each of the equilibria to be even, but that is the same as asking for $|\{i \in L \mid y_i \neq x_i\}|$ to be even. $\square$

A direct corollary of Lemma 3.4 is that a single equilibrium guarantees the existence of many more. In particular, if we can find an equilibrium that contains more than 1 pure strategy, then we get a whole set of equilibria.

Corollary 3.5. *Let $L \subseteq [n]$ and $x = (x_1, \dots, x_n) \in [0, 1]^n$ where $x_i \in \{0, 1\}$ for $i \in L$. Suppose x is a Nash equilibrium with $|L| \geq 1$. Then there are at least $2^{|L|-1}$ Nash equilibria sharing the same mixed strategy profile.*

Proof. This follows directly from Lemma 3.4. Given a Nash equilibrium x , we can obtain another Nash equilibrium by changing the pure strategies of an even number of players in L , while leaving the mixing players unchanged. Since each player in L either changes their strategy or does not, there are $2^{|L|}$ possible strategy profiles on L in total, and exactly half of these differ from pure strategies of x in an even number of positions, giving $2^{|L|-1}$ Nash equilibria. $\square$

3.1.3 Maximal Bound

Now, suppose we fix the number of pure players $|L| = l$, then if we can find a Nash equilibrium for every choice of which l players play pure strategies, then from Corollary 3.5 we get

$$\binom{n}{l} 2^{l-1}!(n-l)$$

number of equilibria, where $\binom{n}{l}$ counts the number of ways to choose which l players play pure strategies, 2^{l-1} is the number of pure strategy profiles from Corollary 3.5, and $!(n-l)$ is the number of derangements of the remaining $n-l$ mixing players. Now, if we instead let the number of pure players vary, then for each $l \geq 1$ we get $\binom{n}{l} 2^{l-1}!(n-l)$ many of equilibria. The resulting number is

$$\sum_{l=1}^n \binom{n}{l} 2^{l-1}!(n-l)$$

and adding the case of a totally mixed equilibrium, it is

$$\sum_{l=1}^n \binom{n}{l} 2^{l-1}!(n-l) + !n. \quad (3.3)$$

This is exactly the Hertling and Vujic upper bound (1.1) on Nash equilibria. This means that as long as we can find a Nash equilibrium for each of the $\binom{n}{l} 2^{l-1}!(n-l)$ cases where $l \geq 1$, then we do achieve the bound. We also observe that the case $l = 0$ is always achieved, as derangements of $[n]$ always exist, giving $!n$ totally mixed equilibria. The case $l = n$ is also straightforward: all players playing 1 is always an equilibrium, and by Corollary 3.5 this yields 2^{n-1} equilibria. It remains to find a Nash equilibrium for each case where $1 \leq l < n$. This is the main result reported in this dissertation.

Chapter 4

Conditions for Maximality

Up to this point, the choice of parameters a_j^i has not played a role in determining whether a strategy profile is an equilibrium. We now shift focus to how this choice affects the number of equilibria. Hertling and Vujic define the delta sort order abstractly through associated permutations δ^j and verify maximality via the increment function. Our approach differs in two respects. We specify the parameters explicitly as $a_j^i = [j - i]_n/n$ and prove maximality by directly analysing the sign of the partial utility function Q_i through a case analysis on Downsteps and Upsteps.

4.1 Illustrative Example

We explore two examples in this section to show how the parameters a_j^i affect the number of equilibria in product two-action games. We start with an example that does achieve the maximal bound, and then we go on to see an example in which the bound isn't achieved.

4.1.1 Case 1: Bound Is Achieved

We take the case of $n = 3$, which has utility functions for each player,

$$\begin{aligned} u^1(x_1, x_2, x_3) &= x_1 & (x_2 - \tfrac{1}{3}) & (x_3 - \tfrac{1}{3}) \\ u^2(x_1, x_2, x_3) &= (x_1 - \tfrac{1}{3}) & x_2 & (x_3 - \tfrac{2}{3}) \\ u^3(x_1, x_2, x_3) &= (x_1 - \tfrac{2}{3}) & (x_2 - \tfrac{2}{3}) & x_3 \end{aligned} \tag{4.1}$$

We now check for all possible equilibria, varying the number of players playing a pure strategy. It is not hard to verify, by substitution or otherwise, that we have 9 equilibria. These equilibria are given in Table 4.1.

0 pure	1 pure	3 pure
$(\frac{2}{3}, \frac{1}{3}, \frac{2}{3})$	$(1, \frac{2}{3}, \frac{2}{3})$	$(1, 1, 1)$
$(\frac{1}{3}, \frac{2}{3}, \frac{1}{3})$	$(\frac{1}{3}, \frac{1}{3}, 1)$	$(1, 0, 0)$
	$(\frac{2}{3}, 0, \frac{1}{3})$	$(0, 1, 0)$
		$(0, 0, 1)$

Table 4.1: All 9 Nash equilibria of the product two-action game.

We now calculate the maximal bound given by equation (3.3)

$$\sum_{l=1}^3 \binom{3}{l} 2^{l-1} l! (3-l)! = 3! + \binom{3}{1} 2^0 1! + \binom{3}{2} 2^1 1! + \binom{3}{3} 2^2 0! = 9$$

We conclude that for the choice of parameters above, we achieve the maximal bound for the case $n = 3$. Further, it turns out that any non-degenerate choice of parameters will achieve this bound. The reason behind this is as follows. We can always find two completely mixed Nash equilibria, as this amounts to finding derangements of 3 elements. In the case of 1 pure player, once the mixing players choose strategies that make another player indifferent, the strategy of the remaining player is always uniquely determined. Finally, in the case of 3 pure players, all players playing 1 is always an equilibrium (as the utility of all players is then positive and no one deviates), and from Lemma 3.4 we are guaranteed four pure equilibria.

So, it seems that the choice of parameters is trivial. However, as we shall see, $n = 3$ marks a special case, and, in general, not all parameters will meet this bound. This is the topic of the next subsection.

4.1.2 Case 2: Bound Is Not Achieved

We build on the preceding discussion and deal with the case of $n = 4$ with utility functions defined as,

$$\begin{aligned} u^1(x_1, x_2, x_3, x_4) &= x_1 - (x_2 - \frac{1}{5}) - (x_3 - \frac{1}{5}) - (x_4 - \frac{3}{5}) \\ u^2(x_1, x_2, x_3, x_4) &= (x_1 - \frac{2}{5}) - x_2 - (x_3 - \frac{3}{5}) - (x_4 - \frac{2}{5}) \\ u^3(x_1, x_2, x_3, x_4) &= (x_1 - \frac{3}{5}) - (x_2 - \frac{4}{5}) - x_3 - (x_4 - \frac{1}{5}) \\ u^4(x_1, x_2, x_3, x_4) &= (x_1 - \frac{4}{5}) - (x_2 - \frac{3}{5}) - (x_3 - \frac{2}{5}) - x_4 \end{aligned} \quad (4.2)$$

We proceed as in Subsection 4.1.1; however, we find that an equilibrium in which players 1 and 2 play pure strategies and players 3 and 4 mix is unattainable. Specifically, if player 4 plays $x_4 = \frac{1}{5}$ and player 3 plays $x_3 = \frac{2}{5}$, then no choice of pure strategies by players 1 and 2 yields an equilibrium. The profile $(1, 1, \frac{2}{5}, \frac{1}{5})$ is not an equilibrium, as $Q_1 < 0$ while player 1 is playing strategy 1, contradicting the best-response condition. The profile $(1, 0, \frac{2}{5}, \frac{1}{5})$ is also not an equilibrium, as player 2 is playing $x_2 = 0$ while their partial utility is positive, so they are not best responding.

This example shows that not every choice of parameters achieves the maximal bound, and that a careful specification is essential.

4.2 Parameter Choice

In the previous section, we saw that the choice of parameters plays a crucial role in determining whether the maximal bound is achieved. A natural question arises: is there a systematic way to choose the parameters a_j^i so that the bound is always attained? It turns out that what matters is the relative ordering of the parameters. This idea is formalised by the *delta sort order*, introduced by Hertling and Vujic, which specifies an ordering of parameters that achieves the maximal bound in the general case. We now define this sort order and show that our explicit parameter specification satisfies it.

4.2.1 Delta Sort Order

The delta sort order is a specific permutation of parameters that achieves the maximal bound. We will first illustrate this sort order with an example and then prove that it works in the general case. The delta order, as defined by Hertling and Vujic, assigns to each player j a unique permutation $\delta^j \in S_{n-1}$ with the following property (Hertling and Vujic, 2024, Lemma 4.2, p. 14, last line): for $j_1, j_2 \in [n] \setminus \{j\}$,

$$\delta^j(j_1) > \delta^j(j_2) \quad \text{if and only if} \quad j_1 \in \{1, \dots, j-1\} \text{ and } j_2 \in \{j+1, \dots, n\}.$$

We believe that the original construction can be simplified by instead presenting the sort order as the permutation $\delta^j \in S_{n-1}$ according to

$$\delta^j = (1, \dots, j-1, j+1, \dots, n) \mapsto (j+1, \dots, n, 1, \dots, j-1). \quad (4.3)$$

That is, δ^j fixes the cyclic order by placing j between the indices larger than j and those smaller than j , while preserving the relative order within each block. To apply the delta sort order to our product-action game, we fix a player $j \in [n]$ and require that the parameters a_j^i are arranged in descending order. Specifically,

$$1 > a_j^{\delta^j(1)} > \dots > a_j^{\delta^j(j-1)} > a_j^{\delta^j(j+1)} > \dots > a_j^{\delta^j(n)} > 0,$$

which, by the definition of δ^j in (4.3), is equivalent to

$$1 > a_j^{j+1} > \dots > a_j^n > a_j^1 > \dots > a_j^{j-1} > 0. \quad (4.4)$$

Note that the permutation for fixed j gives us the ordering of parameters in descending order. We display the permutation for the case of $n = 5$ in Table 4.2.

j	Ordering
1	$a_1^2 > a_1^3 > a_1^4 > a_1^5$
2	$a_2^3 > a_2^4 > a_2^5 > a_2^1$
3	$a_3^4 > a_3^5 > a_3^1 > a_3^2$
4	$a_4^5 > a_4^1 > a_4^2 > a_4^3$
5	$a_5^1 > a_5^2 > a_5^3 > a_5^4$

Table 4.2: Delta sort order for $n = 5$.

We can then write the utility functions explicitly by specifying parameters that follow the ordering. For the case $n = 5$,

$$\begin{aligned}
u^1(x_1, x_2, x_3, x_4, x_5) &= x_1 (x_2 - 0.3) (x_3 - 0.5) (x_4 - 0.7) (x_5 - 0.9) , \\
u^2(x_1, x_2, x_3, x_4, x_5) &= (x_1 - 0.9) x_2 (x_3 - 0.3) (x_4 - 0.5) (x_5 - 0.7) , \\
u^3(x_1, x_2, x_3, x_4, x_5) &= (x_1 - 0.7) (x_2 - 0.9) x_3 (x_4 - 0.3) (x_5 - 0.5) , \\
u^4(x_1, x_2, x_3, x_4, x_5) &= (x_1 - 0.5) (x_2 - 0.7) (x_3 - 0.9) x_4 (x_5 - 0.3) , \\
u^5(x_1, x_2, x_3, x_4, x_5) &= (x_1 - 0.3) (x_2 - 0.5) (x_3 - 0.7) (x_4 - 0.9) x_5 .
\end{aligned}$$

An intuitive way to think about this sort order is in terms of columns and rows. We place the parameters in descending order in the first column, then cyclically shift the last row entry to the first row in the next column, continuing for all columns.

While the delta sort order has a clean combinatorial description, it is not immediately obvious that it achieves the maximal bound for all n . To verify this, we wrote the program given in Appendix A.1. The program generates parameters a_j^i that follow the delta sort order and verifies that the bound (3.3) is achieved for any value of n . It also reveals structural patterns that guided the proof strategy presented in the next section.

4.3 Sort-Order Proof

4.3.1 π Notation

We start by observing that any strategy profile of an n -player product two-action game can be described completely by a permutation $\pi \in S_n$. As each mixing player corresponds to an index that is deranged, and the pure players correspond to fixed points. We write a permutation in Cauchy two-line notation (Wussing, 2007, p. 94)

$$\begin{pmatrix} 1 & 2 & \dots & n \\ \pi(1) & \pi(2) & \dots & \pi(n) \end{pmatrix} \quad (4.5)$$

For our product two-action game, we interpret the first row of (4.5) as the players who choose a strategy, and the second row as the players who are made indifferent as a result of this choice. To distinguish mixing and pure players, we define,

$$M(\pi) = \{j \in [n] : \pi(j) \neq j\} \text{ and } P(\pi) = \{j \in [n] : \pi(j) = j\}. \quad (4.6)$$

The strategy profile can now be written explicitly in terms of the parameters. A mixing player $j \in M(\pi)$ makes some other mixing player i indifferent, which is exactly captured by the derangement $\pi(j) = i$. As a consequence, player j 's strategy is $x_j = a_j^{\pi(j)} = a_j^i$.

4.3.2 Parameter Specification

To establish that the bound is achieved, it suffices to exhibit a single specification of parameters that attains it. Let $[x]_n$ denote x taken modulo n . We define our parameters by

$$a_j^i = \frac{[j - i]_n}{n}.$$

and verify that this specification follows the delta sort order. For each $j \in [n]$, we can check whether (4.4) is satisfied. The delta sort order in terms of parameters is

$$1 > a_j^{j+1} > \dots > a_j^n > a_j^1 > \dots > a_j^{j-1} > 0.$$

Substituting our parameter values gives

$$1 > \frac{[j - (j+1)]_n}{n} > \dots > \frac{[j - n]_n}{n} > \frac{[j - 1]_n}{n} > \dots > \frac{[j - (j-1)]_n}{n} > 0.$$

Equivalently,

$$1 > \frac{n-1}{n} > \dots > \frac{j}{n} > \frac{j-1}{n} > \dots > \frac{1}{n} > 0.$$

Since this holds for all $j \in [n]$, the delta sort order is satisfied.

4.3.3 Case Analysis and Main Result

We start by defining some terms that become relevant in our proof,

Definition 4.1. Define set $D(\pi) = \{i \in [n] : \pi(i) < i\}$. We call an index a Downstep if it belongs to $D(\pi)$. Similarly, an Upstep is an element of the set $U(\pi) = \{i \in [n] : \pi(i) > i\}$.

We note that Upsteps and Downsteps correspond to exceedances ($\pi(i) > i$) and anti-exceedances ($\pi(i) < i$) as studied in the combinatorics literature Mantaci (1991); Chen (2019).

Suppose we have some permutation π that describes a strategy profile of a product two-action game. We begin by making some observations regarding the pure players. Note that if $i \in P(\pi)$ (defined in (4.6)), then $i \notin D(\pi)$ and $i \notin U(\pi)$, since $\pi(i) = i$ by definition. For a pure player i and a Downstep $j \in D(\pi)$, there are three possible configurations of i , j , and $\pi(j)$:

1. $j > i$ and $\pi(j) > i$, so that $i < \pi(j) < j$.
2. $j > i$ and $\pi(j) < i$, so that $\pi(j) < i < j$.

3. $j < i$, so that $\pi(j) < j < i$.

We now suppose that the parameters follow the specification in (4.3.2) and examine the decomposition of the partial utility function for a pure player i .

$$Q_i(x_{-i}) = \prod_{j \in P(\pi) \setminus \{i\}} (x_j - a_j^i) \prod_{j \in M(\pi)} (x_j - a_j^i). \quad (4.7)$$

We make the simplifying assumption that all pure players play strategy 1, giving us

$$\text{sign}(Q_i(x_{-i})) = \text{sign} \left(\prod_{j \in M(\pi)} (x_j - a_j^i) \right). \quad (4.8)$$

Clearly, the factors $(a_j^{\pi(j)} - a_j^i)$ are either positive or negative; our next step is to count how many of each there are. From our discussion in Subsection 4.3.1, we know that for a mixing player $j \in M(\pi)$, the strategy is given by $x_j = a_j^{\pi(j)}$. We now examine the sign of the factor $a_j^{\pi(j)} - a_j^i$ for each of the three configurations.

Now, following our parameter specification in subsection 4.3.2, we analyse the various cases.

For a Downstep at j (so $\pi(j) < j$), the factor is

$$a_j^{\pi(j)} - a_j^i = \frac{[j - \pi(j)]_n - [j - i]_n}{n}.$$

Since $\pi(j) < j$, we have $[j - \pi(j)]_n = j - \pi(j)$.

Case 1: $i < \pi(j) < j$. Then $[j - i]_n = j - i > j - \pi(j) = [j - \pi(j)]_n$. Factor is **negative**.

Case 2: $\pi(j) < i < j$. Then $[j - i]_n = j - i < j - \pi(j) = [j - \pi(j)]_n$. Factor is **positive**.

Case 3: $\pi(j) < j < i$. Then $[j - i]_n = j - i + n > j - \pi(j) = [j - \pi(j)]_n$. Factor is **negative**.

For an Upstep at j (so $\pi(j) > j$), we have $[j - \pi(j)]_n = j - \pi(j) + n$.

Case 1: $j < \pi(j) < i$. Then $[j - i]_n = j - i + n < j - \pi(j) + n = [j - \pi(j)]_n$. Factor is **positive**.

Case 2: $j < i < \pi(j)$. Then $[j - i]_n = j - i + n > j - \pi(j) + n = [j - \pi(j)]_n$. Factor is **negative**.

Case 3: $i < j < \pi(j)$. Then $[j - i]_n = j - i < j - \pi(j) + n = [j - \pi(j)]_n$. Factor is **positive**.

These six cases exhaust all possibilities. Finding equilibria now reduces to determining the sign of the partial utility function Q_i , which in turn requires counting the number of negative factors in (4.8). It turns out this can be done. We begin with an observation regarding the two Case 2s above.

Lemma 4.2. *Given permutation π define sets $M = \{i \in [n] : \pi(i) \neq i\}$ and $P = \{i \in [n] : \pi(i) = i\}$, then for any $i \in P$ we have,*

$$|\{j \in M : \pi(j) < i < j\}| = |\{j \in M : j < i < \pi(j)\}|.$$

Proof. See A.2.1 □

Lemma 4.2 allows us to count the negative factors of Q_i directly from the number of Downsteps. In set notation, we have

$$\begin{aligned} D(\pi) &= \underbrace{\{j : i < \pi(j) < j\}}_{\text{Case 1: -ve factors of } Q_i} \cup \underbrace{\{j : \pi(j) < i < j\}}_{\text{Case 2: +ve}} \cup \underbrace{\{j : \pi(j) < j < i\}}_{\text{Case 3: -ve}} \\ \Rightarrow |D(\pi)| &= |\{j : i < \pi(j) < j\}| + |\{j : \pi(j) < i < j\}| + |\{j : \pi(j) < j < i\}|. \end{aligned}$$

But by Lemma 4.2, we have

$$|D(\pi)| = \underbrace{|\{j : i < \pi(j) < j\}|}_{\text{Case 1: -ve factors}} + \underbrace{|\{j : j < i < \pi(j)\}|}_{\text{Upstep Case 2: -ve factors}} + \underbrace{|\{j : \pi(j) < j < i\}|}_{\text{Case 3: -ve factors}} \quad (4.9)$$

Note that calling a set A , the negative factors of Q_i means that the players in A play strategy $x_j = a_j^{\pi(j)}$ such that the term $(x_j - a_j^i)$ is negative. So the immediate result we have is a corollary regarding the sign of Q_i

Corollary 4.3. *Given a permutation π , for any $i \in P(\pi)$, suppose $x_j = 1$ for all $j \in P(\pi) \setminus \{i\}$, i.e., all pure players other than i play 1, then the sign of the partial utility function is given by the equation*

$$\text{sign}(Q_i(x_{-i})) = -1^{|D(\pi)|}.$$

Proof. Since all pure players other than i play $x_j = 1$, the partial utility function decomposes as

$$Q_i(x_{-i}) = \prod_{j \in P(\pi) \setminus \{i\}} (1 - a_j^i) \cdot \prod_{j \in M(\pi)} (a_j^{\pi(j)} - a_j^i).$$

Each factor in the first product is positive since $a_j^i \in (0, 1)$. By equation (4.9), the second product has exactly $|D(\pi)|$ negative factors. Therefore $\text{sign}(Q_i(x_{-i})) = (-1)^{|D(\pi)|}$. □

Having done the main heavy lifting by analysing the cases and observing the resulting structure, we now turn to the main result: proving that for any permutation π , we can always construct a Nash equilibrium.

Theorem 4.4. *For the product two-action game with coefficients $a_j^i = \frac{[j-i]_n}{n}$, if we are given any permutation π with fixed point set $P(\pi) \neq \emptyset$, the strategy profile x defined by:*

- $x_j = a_j^{\pi(j)}$ for $j \in M(\pi)$,
- $x_i = (|D(\pi)| + 1) \bmod 2$ where $i = \min P(\pi)$,
- $x_k = 1$ for $k \in P(\pi) \setminus \{\min P(\pi)\}$,

is a Nash equilibrium.

Proof. Part 1: Mixing players are indifferent.

For $j \in M(\pi)$, player j plays $x_j = a_j^{\pi(j)} \in (0, 1)$, which is the strategy that makes player $\pi(j)$ indifferent between their two actions.

Part 2: Pure players are best responding.

The best response for a pure player i is determined by the sign of

$$Q_i(x_{-i}) = \prod_{j=1, j \neq i} (x_j - a_j^i).$$

Player i plays 1 if $Q_i > 0$ and 0 if $Q_i < 0$. We decompose $Q_i(x)$ as

$$Q_i(x_{-i}) = \underbrace{\prod_{j \in P(\pi) \setminus \{i\}} (x_j - a_j^i)}_{\text{Pure players}} \cdot \underbrace{\prod_{j \in M(\pi)} (a_j^{\pi(j)} - a_j^i)}_{\text{Mixing players}}.$$

Since every other pure player plays 1, and $1 > a_j^i$ for all $j \neq i$, each such factor $(1 - a_j^i)$ is positive. This leaves us with

$$Q_i(x_{-i}) = \prod_{j \in M(\pi)} (a_j^{\pi(j)} - a_j^i). \quad (4.10)$$

So by Corollary 4.3 we have that

$$\text{sign}(Q_i(x_{-i})) = -1^{|D(\pi)|} \quad (4.11)$$

Now by Theorem 3.3 we know that player i is best responding when they play 1 when $|D(\pi)|$ is even and 0 when $|D(\pi)|$ is odd, i.e. $x_i = (|D(\pi)| + 1) \bmod 2$.

Finally, we need to check whether every other pure player is also best responding. Suppose $i = \min P(\pi)$. Now we take some $k \in P(\pi) \setminus \{i\}$, and analyse the partial utility function. We need $Q_k(x_{-k}) > 0$ to confirm $x_k = 1$ is the best response.

$$Q_k(x_{-k}) = (x_i - a_i^k) \cdot \prod_{j \in P(\pi) \setminus \{k, i\}} (x_j - a_j^k) \cdot \prod_{j \in M(\pi)} (a_j^{\pi(j)} - a_j^k) \Rightarrow$$

$$\text{sign}(Q_k(x_{-k})) = \text{sign}(x_i - a_i^k) \cdot (+1)^{|P(\pi)|-2} \cdot (-1)^{|D(\pi)|} = \text{sign}(x_i - a_i^k) \cdot (-1)^{|D(\pi)|}.$$

Note that Corollary 4.3 cannot be applied directly to player k , since player i may play 0; however, the factor $(x_i - a_i^k)$ is isolated and its sign handled separately in the case analysis below. *Case A: $x_i = 1$, i.e. $|D(\pi)|$ is even.* Then $\text{sign}(x_i - a_i^k) = +1$, so

$$\text{sign}(Q_k(x_{-k})) = (+1) \cdot (-1)^{|D(\pi)|} = +1.$$

Best response is $x_k = 1$.

Case B: $x_i = 0$, i.e. $|D(\pi)|$ is odd. Then $\text{sign}(x_i - a_i^k) = -1$, so

$$\text{sign}(Q_k(x_{-k})) = (-1) \cdot (-1)^{|D(\pi)|} = (-1)^{|D(\pi)|+1} = +1.$$

Best response is $x_k = 1$.

Every fixed point is best responding, and every mixing player is indifferent, so π defines a Nash equilibrium. $\square$

We have shown that in any equilibrium, the mixing players play some strategy in $(0, 1)$ while the pure players play either 0 or 1. For each permutation π , the equilibrium takes one of two forms: either all pure players play 1, or the first pure player plays 0, and the rest play 1. Which case applies is determined by the number of Downsteps $|D(\pi)|$. If $|D(\pi)|$ is even, then all pure players play 1; if $|D(\pi)|$ is odd, then the first pure player plays 0, and the rest play 1.

Furthermore, if there is one equilibrium that involves l pure players where $l > 0$, any other such equilibrium is found by changing any two of these players switching their strategy, which gives 2^{l-1} equilibria in total, by Lemma 3.4(d) and Corollary 3.5.

Following our discussion in Subsection 3.1.3, having constructed a Nash equilibrium for each of the non-trivial cases $1 \leq l < n$, we conclude that the bound is achieved.

Chapter 5

Conclusion

This dissertation has presented an accessible account of the maximal number of Nash equilibria in product two-action games, building on the framework of Hertling and Vujic (2024). Our primary contribution is a cleaner and more direct proof that the upper bound (3.3) is achievable. Rather than using the increment function of Hertling and Vujic (2024), we argue directly via the best-response condition of Theorem 3.3, reducing the problem of finding equilibria to determining the sign of the partial utility function Q_i . A key insight of our proof is that the question of whether a permutation yields a Nash equilibrium reduces to counting the number of Downsteps in that permutation. Corollary 4.3 establishes this very link between the combinatorics of permutations and the equilibrium structure of the game. This connection is not apparent from the framework of Hertling and Vujic (2024), where the analysis proceeds through an increment function that obscures the underlying combinatorial structure. The proof strategy of Theorem 4.4 was itself guided by computational experiments: The implementation in Appendix A.1 revealed that the number of Downsteps consistently determined the equilibrium structure, which motivated the case analysis that followed.

A natural direction for future work is to consider games with more than two strategies per player. McKelvey and McLennan (1997) describe a construction of such games with the maximum number of *completely* mixed equilibria in their equation (4.1). It has a similar product structure to (3.1) with a multilinear expression in each player's probabilities instead of the single-probability term $(x_j - a_j^i)$. The question is how to extend this to the players who play pure strategies in order to maximize the number of all equilibria, via a suitable choice of the parameters for the multilinear expressions. Alternatively, one could retain the two-action restriction but drop the product structure, and ask how many equilibria are possible in a game where each player has exactly two strategies, as was done for $n = 3$ by Jahani and von Stengel (2022). We hope that this dissertation provides an accessible entry point for those interested in the maximal number of Nash equilibria.

Bibliography

- Chen, R. X. F. (2019). Separation probabilities and analogues of a Zagier-Stanley formula. URL <https://arxiv.org/abs/1909.13388>.
- Hertling, C. and M. Vujic (2024). Maximal number of mixed Nash equilibria in generic games where each player has two pure strategies. URL <https://arxiv.org/abs/2412.17890>.
- Ickstadt, C., T. Theobald, and B. von Stengel (2025). A stable-set bound and maximal numbers of Nash equilibria in bimatrix games. *Mathematics of Operations Research Articles in Advance*.
- Jahani, S. and B. von Stengel (2022). Automated equilibrium analysis of $2 \times 2 \times 2$ games. In: *Algorithmic Game Theory. SAGT 2022*, edited by P. Kanellopoulos, M. Kyropoulou, and A. Voudouris, volume 13584 of *Lecture Notes in Computer Science*, 223–237. Springer, Cham.
- Keiding, H. (1997). On the maximal number of Nash equilibria in an $n \times n$ bimatrix game. *Games Econ. Behav.* 21(1-2), 148–160.
- Lemke, C. E. and J. T. Howson, Jr. (1964). Equilibrium points of bimatrix games. *Journal of the Society for Industrial and Applied Mathematics* 12(2), 413–423.
- Mantaci, R. (1991). Anti-exceedances in permutation groups. *European Journal of Combinatorics* 12(3), 237–244.
- McKelvey, R. D. and A. McLennan (1997). The maximal number of regular totally mixed Nash equilibria. *Journal of Economic Theory* 72(2), 411–425.
- McLennan, A. and I.-U. Park (1999). Generic 4×4 two person games have at most 15 Nash equilibria. *Games Econ. Behav.* 26(1), 111–130.
- Nash, J. (1951). Non-cooperative games. *Annals of Mathematics* 54(2), 286–295.
- Shubik, M. and T. Quint (1998). A theorem on the number of Nash equilibria in a bimatrix game. *International Journal of Game Theory* 26, 353–359.

- Stanley, R. P. (2011). *Enumerative Combinatorics*, volume 1. Cambridge University Press, second edition.
- von Stengel, B. (1999). New maximal numbers of equilibria in bimatrix games. *Discrete and Computational Geometry* 21, 557–568.
- von Stengel, B. (2022). *Game Theory Basics*. Cambridge University Press, Cambridge, UK.
- Wussing, H. (2007). *The Genesis of the Abstract Group Concept: A Contribution to the History of the Origin of Abstract Group Theory*. Dover Publications, Mineola, NY.

Appendix A

A.1 Code

The code below computes all equilibria for any n using the parameter specification $a_j^i = [j-i]_n/n$. For each equilibrium, the code also displays the number of Downsteps and Upsteps, concepts which become central to the proof of Theorem 4.4. It was through observing the pattern of Downsteps and Upsteps in the output of this code that we were motivated to pursue the proof strategy presented in this dissertation.

```
import numpy as np
from itertools import permutations
from itertools import combinations

class ProductTwoActionGame:
    def __init__(self, n):
        self.n = n
        self.A = self._generate_coefficients()

    def _generate_coefficients(self):
        A = np.zeros((self.n, self.n))
        for i in range(self.n):
            for j in range(self.n):
                A[i, j] = ((j - i) % self.n) / self.n
        return np.round(A, 2)

    def get_derangements_with_fixed_points(self, num_fixed):
        from itertools import combinations
        elements = tuple(range(1, self.n + 1))
        results = []

        for fixed_positions in combinations(range(self.n), num_fixed):
            non_fixed_positions = [i for i in range(self.n) if i not in fixed_positions]
            non_fixed_elements = [elements[i] for i in non_fixed_positions]

            for perm in permutations(non_fixed_elements):
                is_derangement = True
                for i, pos in enumerate(non_fixed_positions):
                    if perm[i] == elements[pos]:
                        is_derangement = False
                        break

                if is_derangement:
                    full_perm = [0] * self.n
                    for pos in fixed_positions:
                        full_perm[pos] = elements[pos]
                    for i, pos in enumerate(non_fixed_positions):
                        full_perm[pos] = perm[i]

                    results.append(tuple(full_perm))

        return results

    def game_matrix(self, x):
        X_matrix = np.tile(x, (self.n, 1))
        M = X_matrix - self.A
        np.fill_diagonal(M, 0)
        return M

    def parity(self, M):
        return (M < 0).astype(int)

    def extract_coefficients(self, pi):
        row_indices = [p - 1 for p in pi]
        col_indices = list(range(len(pi)))
        return self.A[row_indices, col_indices]

    def possible_equilibria(self):
        all_perms = []
        for num_fixed in range(self.n + 1):
            perms = self.get_derangements_with_fixed_points(num_fixed)
            all_perms.extend(perms)
        return all_perms
```

```

def equilibria(self):
    all_perms = self.possible_equilibria()
    equilibria_list = []

    for pi in all_perms:
        x = self.extract_coefficients(pi)
        M = self.game_matrix(x)
        P = self.parity(M)
        row_sums = np.sum(P, axis=1) % 2
        inc = (1 + x + row_sums) % 2

        integers = [float(val) for val in inc if val == int(val)]

        modified_coeffs = x.copy()

        if 1.0 in integers:
            for i in range(len(modified_coeffs)):
                if modified_coeffs[i] == 0.0:
                    modified_coeffs[i] = 1.0
                    break

        equilibria_list.append(modified_coeffs)

    return equilibria_list

def partition_equilibria(self):
    print(self.A)
    equilibria_list = self.equilibria()

    partitions = {i: [] for i in range(self.n + 1)}

    for eq in equilibria_list:
        num_decimals = sum(1 for val in eq if val != int(val))
        partitions[num_decimals].append(eq)

    print(f"\nPartitioning {len(equilibria_list)} equilibria by mixing players:\n")
    print("="*70)

    for num_dec in range(self.n, -1, -1):
        results = partitions[num_dec]
        print(f"\n{num_dec} mixed ({len(results)} equilibria):")
        print("="*70)
        for eq in results:
            print(f"    {eq}")

    print("="*70)

    return partitions

def equilibrium_count(self):
    equilibria_list = self.equilibria()

    partitions = {i: [] for i in range(self.n + 1)}

    for eq in equilibria_list:
        num_decimals = sum(1 for val in eq if val != int(val))
        partitions[num_decimals].append(eq)

    total_count = 0

    print(f"\nEquilibrium Count Formula:\n")
    print("="*70)

    count_n = len(partitions[self.n])
    total_count += count_n
    print(f"{self.n} mixed (completely mixed): {count_n}")

    for num_mixed in range(self.n - 1, -1, -1):
        count = len(partitions[num_mixed])
        weight = 2 ** (self.n - num_mixed - 1)
        contribution = count * weight
        total_count += contribution

        print(f"{num_mixed} mixed: {count} * 2^{self.n-(num_mixed)-1} = {count} * {weight} = {contribution}")

    print("="*70)
    print(f"Total equilibria count: {total_count}\n")

    return total_count

def equilibrium_count_1(self):
    equilibria_list = self.equilibria()

    partitions = {i: [] for i in range(self.n + 1)}
    for eq in equilibria_list:
        num_decimals = sum(1 for val in eq if val != int(val))
        partitions[num_decimals].append(eq)

    total_count = 0

    print(f"\nEquilibrium Count (n = {self.n}):")
    print("=" * 70)

    num_mixed = self.n
    eqs = partitions[num_mixed]
    total_count += len(eqs)

    print(f"\n0 Pure, {self.n} Mixed: 1{self.n} = {len(eqs)}")
    print("-" * 70)
    if len(eqs) == 0:
        print("    (none)")
    else:
        for x in eqs:
            print(f"    {x}")
    print("-" * 70)

    for l in range(1, self.n + 1):
        num_mixed = self.n - l
        eqs = partitions[num_mixed]

```

```

from math import comb, factorial
def subfactorial(k):
    if k == 0:
        return 1
    return int(round(factorial(k) * sum((-1)**i / factorial(i) for i in range(k + 1))))

derangements = subfactorial(num_mixed)
binom = comb(self.n, 1)
pure_combos = 2 ** (1 - 1)
contribution = binom * pure_combos * derangements
total_count += contribution

print(f"\n{1} Pure, {num_mixed} Mixed: C({self.n},{1}) * 2^{(1)-1} * !(num_mixed) = {binom} * {pure_combos} *
↪ {derangements} = {contribution}")
print("-" * 70)
if len(eqs) == 0:
    print("    (none)")
else:
    for x in eqs:
        print(f"    {x}")
print("-" * 70)

print("\n" + "=" * 70)
print(f"Total: C({self.n},1) * 2^{(1)-1} * !({self.n}-1) = {total_count}\n")

return total_count

def count_Upsteps(self, pi):
    up = sum(1 for i in range(len(pi)) if pi[i] > i + 1)
    down = sum(1 for i in range(len(pi)) if pi[i] < i + 1)
    return up, down

def display_equilibria_compact(self):
    equilibria_list = self.equilibria()

    print(f"\nEquilibria Display:\n")
    print("=" * 70)
    print("Coefficient Matrix A:")
    for row in self.A:
        print(f"    {row}")
    print("=" * 70)

    count = 0
    for x in equilibria_list:
        num_mixed = sum(1 for val in x if val != int(val))

        if num_mixed == self.n:
            continue

        pi = []
        for j in range(self.n):
            if x[j] == int(x[j]):
                pi.append(j + 1)
            else:
                for i in range(self.n):
                    if i != j and np.isclose(self.A[i, j], x[j]):
                        pi.append(i + 1)
                        break

        up, down = self.count_Upsteps(pi)

        count += 1
        print(f"\n{count}. pi = {pi} | Upsteps: {up} Downsteps: {down}")
        print(f"    Coeffs: {x}")

    print("\n" + "=" * 70)
    print(f"Total displayed: {count} equilibria\n")

#-----Usage starts here
# The method display_equilibria_compact shows equilibria for cases n-1, n-2, ..., 0 mixing players, along with the permutation
↪ and the number of Upsteps and Downsteps.
# The method equilibrium_count gives a summary count of equilibria by number of mixing players.
# The method equilibrium_count_1 gives a detailed breakdown showing each equilibrium
# partitioned by number of pure and mixing players, verifying the bound is achieved.

n = 5
game = ProductTwoActionGame(n)
print(game.display_equilibria_compact())
print(game.equilibrium_count_1())

```

A.2 Proofs

A.2.1 Case 2 Cardinality

We note that π is a bijection, as it is an element of the symmetric group S_n . Since $i \in P$, the permutation π restricts to a bijection on M . Define $A = \{j \in M : j > i\}$ and $B = \{j \in M : j < i\}$, so that $M = A \cup B$. Since π restricts to a bijection on M , the number of elements of M that are mapped above i equals the number of elements of M that lie above i , which is $|A|$. These come from two sources: elements

of A that map above i , and elements of B that map above i . That is,

$$|A| = |\{j \in A : \pi(j) > i\}| + |\{j \in B : \pi(j) > i\}|.$$

On the other hand, partitioning A by where its elements map gives

$$|A| = |\{j \in A : \pi(j) > i\}| + |\{j \in A : \pi(j) < i\}|,$$

where $\pi(j) \neq i$ since i is a fixed point. Comparing these two expressions, we obtain

$$|\{j \in A : \pi(j) < i\}| = |\{j \in B : \pi(j) > i\}|,$$

which is the desired result.